Threatopia API Documentation

1. Overview

This document describes the **Threatopia platform API**.
It defines the communication contract between the **Frontend (Next.js)** and the **Backend (Node.js / Express)**.

The API allows the frontend to:

- Register and authenticate users
- Access cybersecurity challenges
- Track user progress
- View leaderboards
- Use the AI security assistant
- Access simulations
- Manage admin operations

According to modern web architecture practices, APIs act as the interface that allows different parts of a system to communicate with each other.

2. API Base Information

Base URL
http://localhost:5000/api
Content Type
`application/json`
Authentication
Most endpoints require authentication using **JWT Bearer Tokens**.
Header format:
`Authorization: Bearer <your-jwt-token>`
JWT tokens are commonly used to secure APIs by verifying user identity and permissions.

3. System & Health Endpoints

These endpoints allow checking the status of the backend system.

Check Server Health

Endpoint
`GET /health`
Description
Verifies that the server is running correctly.
Response

```
200 OK
{
  "status": "ok",
  "message": "Server is running"
}
```

Maintenance Mode

Endpoint

`GET /maintenance`

Description

Checks if the platform is currently in maintenance mode.

Status

🚧 Coming Soon

4. Authentication

Handles **user registration, login, and session management**.

Register User

Endpoint

`POST /auth/register`

Request Body

```
{
  "name": "string",
  "email": "string",
  "password": "string",
  "ageGroup": "teen | student | professional"
}
```

Success Response

```
201 Created
{
  "success": true,
  "data": {
    "user": {
      "id": "user_id",
      "name": "User Name"
    },
    "token": "jwt-token"
  }
}
```

Login User

Endpoint

`POST /auth/login`

Request Body

```
{
  "email": "string",
  "password": "string"
}
```

Success Response

```
200 OK
{
  "success": true,
  "data": {
    "user": {
      "id": "user_id",
      "role": "student"
    },
    "token": "jwt-token"
  }
}
```

5. Challenges & Simulations

These endpoints provide the **educational cybersecurity content** of the platform.

Get All Challenges

Endpoint

`GET /challenges`

Description

Returns a list of all available challenges.
Challenges can be filtered by:

- category
- difficulty

Status

✅ Working

Get Challenge Details

Endpoint

`GET /challenges/:id`

Description

Returns full information about a specific challenge.
Example:

`GET /challenges/123`

Status

✅ Working

Submit Challenge Attempt

Endpoint

`POST /challenges/:id/attempt`

Description

Submit an answer for validation.

Status

🚧 Coming Soon

Response Example

```
{
  "success": true,
  "data": {
    "correct": true,
    "pointsEarned": 50,
    "explanation": "Great job! SQL Injection happens when
inputs aren't sanitized.",
    "nextChallengeId": "id_123"
  }
}
```

6. Simulations

Interactive cybersecurity labs used for real-world practice.

Get Available Simulations

Endpoint

`GET /simulations`

Description

Returns metadata about available simulation labs.
Example fields may include:

- simulation id
- title
- difficulty
- description

Status

🚧 Coming Soon

7. User Profile & Progress

These endpoints track user learning progress and achievements.

Get User Profile

Endpoint

`GET /users/profile`

Description
Returns user progress information including points, badges, and completed challenges.
Response Example
```
{
  "success": true,
  "data": {
    "points": 1250,
    "level": 5,
    "badges": [
      "Phishing Hunter",
      "First Login"
    ],
    "completedChallenges": [
      "id1",
      "id2"
    ],
    "stats": {
      "sqliScore": 80,
      "phishingScore": 100
    }
  }
}
```

Get Notifications

Endpoint
`GET /users/notifications`
Description
Returns notifications such as:
- leaderboard changes
- new challenges
- system alerts

8. Gamification – Leaderboard

The leaderboard encourages competition between users.

Get Leaderboard

Endpoint
`GET /leaderboard`
Query Parameters
`?type=global`
`?type=friends`
Response Example

```
{
  "success": true,
  "data": [
    {
      "rank": 1,
      "name": "Omar",
      "points": 5000,
      "avatar": "url"
    },
    {
      "rank": 2,
      "name": "Sara",
      "points": 4800,
      "avatar": "url"
    }
  ]
}
```

9. AI Security Assistant

Provides intelligent cybersecurity guidance and explanations.

Chat with AI Assistant

Endpoint

```
POST /assistant/chat
```

Request Body

```
{
  "message": "How can I prevent SQL Injection?"
}
```

Response Example

```
{
  "success": true,
  "data": {
    "reply": "To prevent SQL Injection, always use prepared
statements.",
    "relatedChallenges": [
      "id_sql_1",
      "id_sql_2"
    ]
  }
}
```

10. Admin Panel

These endpoints are accessible only to **administrators or instructors**.

Platform Analytics

Endpoint

`GET /admin/analytics`

Provides platform statistics such as:

- total users
- active users
- average scores

Manage Users

Endpoint

`GET /admin/users`

Returns a list of all registered users.

Admins can manage:

- roles
- permissions
- accounts

Create Challenge

Endpoint

`POST /admin/challenges`

Allows admins to create a new challenge or simulation task.

Generate User Report

Endpoint

`GET /admin/reports/:userId`

Generates a performance report for a specific user.

Example:

`GET /admin/reports/123`

11. Error Handling

The API returns standard HTTP error codes.

Code	Meaning
400	Bad Request – Invalid input
401	Unauthorized – Missing or invalid token
403	Forbidden – User lacks permission
404	Resource not found
500	Internal server error

Example error response:

```
{
  "success": false,
  "error": "Invalid email or password"
}
```

12. Development Workflow

Recommended workflow between frontend and backend teams:

- Backend builds endpoints marked as **Coming Soon**.
- Frontend uses **Mock Data Services** until APIs are ready.
- Once an API is completed, the frontend replaces the mock service with a real API call.
- All responses must follow the same **JSON structure** defined in this document.